

Portfolio

데이터를 잃지 않는 흐름을 설계하고, 테스트로 검증하는 백엔드 개발자

Java · Spring Boot · IoT 센서 데이터 파이프라인 · FastAPI 예측 서비스 연동 · Spring AI · 통계 전공

권세윤

010-3616-4913 · tpdbs3616@naver.com

<https://github.com/sy-103>

Index

01 About Me

이력 · 자격 · 기술

02 Keyword

저를 설명하는 네 가지

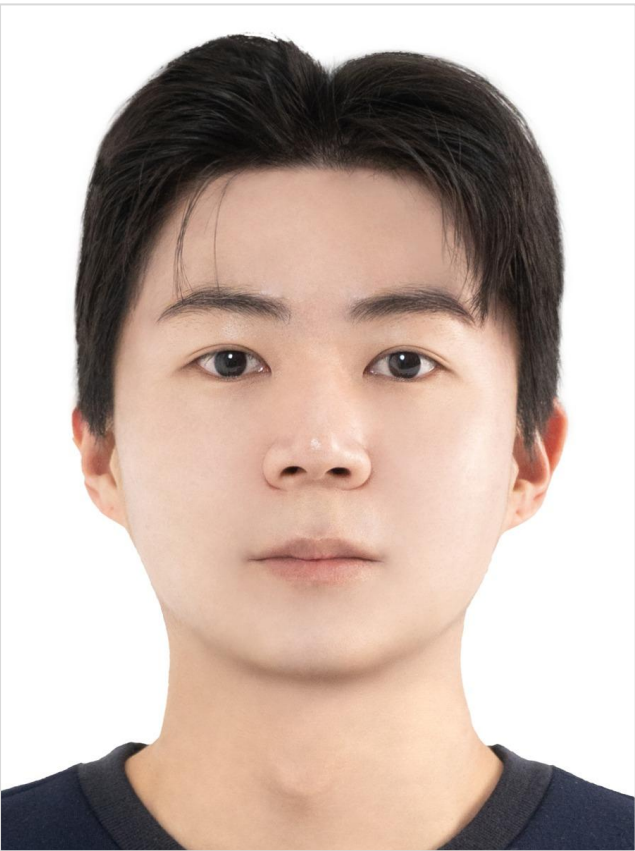
03 Project

omagotchi · Spring AI Library · Mini Dooray

04 Data Project

범죄 심각도 지수 · 공부시간 예측 · 사계절 붕괴와 에너지 수요

About Me



권세윤

Gwon Seyun

1999.03.24 · 010-3616-4913 · tpdb3616@naver.com · <https://github.com/sy-103>

데이터를 잃지 않는 흐름을 만드는 백엔드 개발자

통계학을 전공하며 데이터를 검증하는 습관을 익혔고, NHN아카데미에서 Java · Spring Boot 백엔드 개발을 배워 IoT 센서 데이터 파이프라인과 FastAPI 예측 서비스 연동을 구현했습니다.

학력	조선대학교 컴퓨터통계학과 졸업 (2018.03 – 2026.02) · 학점 3.73 / 4.5
교육 · 경력	NHN아카데미 광주 AIoT 3기 인공지능 기반 IoT 웹 서비스 개발자 과정 (2025.12 ~ 2026.09) 조선대학교 빅데이터분석실습실 실습실원 (2024.03 ~ 2026.02)
수상 · 논문	KCI 등재지 통합자연과학논문집 제2저자 (2026.02) 한국통계학회 학회장상 (2026.02) 데이터분석 경진대회 금상(총장상) 기후위기대응 데이터분석 경진대회 금상
활동	컴퓨터통계학과 학회 회장 (2023) · 환경동아리 Fortrees 회장 (2024) · 글로벌 그린 챌린저 GGC (2025)

기술 스택

- Backend
 - Java · Spring Boot · JPA · Spring AI
- Data
 - MySQL · PostgreSQL · InfluxDB
- Messaging
 - MQTT · RabbitMQ
- Test
 - JUnit 5 · Mockito · JaCoCo
- Tools
 - Git · Docker
- Analytics
 - Python · R · SPSS

자격증

ADsP · SQLD

Keyword

유실 없는 흐름

데이터가 들어와 저장될 때까지 한 건도 잃지 않도록 설계하고 보낸 수와 받은 수를 맞춰 확인합니다.

원인까지 추적

문제가 생기면 데이터가 지나온 경로를 하나씩 따라가 틀어진 지점을 찾아 고칩니다.

테스트로 증명

기능을 만들면 실패하는 경우까지 테스트로 확인합니다. 기다려서는 볼 수 없는 불량 데이터는 시뮬레이터로 재현했습니다.

결정을 기록

왜 그렇게 설계했는지 설계 결정 기록(ADR)으로 남겨 팀원이 같은 고민을 반복하지 않게 합니다.

Project 01

AI 기반 학습 관리 서비스 (omagotchi)

8인 MSA 프로젝트

2026.07 ~ 2026.09 · 8인

발표 영상 보기

<https://youtu.be/fns6UE-Ww8Q>

배포 사이트 보기

<https://omagotchi.site/>

GitHub

<https://github.com/nhnacademy-aiot3-omagotchi>

맡은 역할

IoT 센서 데이터 파이프라인

수집 · 정규화 · 품질 판정 · 임계값 규칙 · 시뮬레이터

예측 퀘스트 연동

FastAPI 예측 서비스 ↔ Spring Boot 퀘스트

AI 코칭 · 센서 대시보드

Spring AI 도구 4종 · InfluxDB 조회 API · 센서 차트

Project

AI 기반 개인 맞춤형 학습 관리 서비스

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

① 개발 배경

- 학습 공간에 설치된 IoT 센서(LoRaWAN) 10대가 온도 · 습도 · CO2 같은 환경 데이터를 보내지만, 실제 센서 데이터에는 결측 · 중복 · 지연 같은 불량이 섞여 들어옵니다.
- 저희 팀은 이 데이터로 학습 공간의 상태를 보여 주고, 공부 기록으로 예측한 내일 공부시간을 퀘스트로 주는 학습 관리 서비스를 만들었습니다.

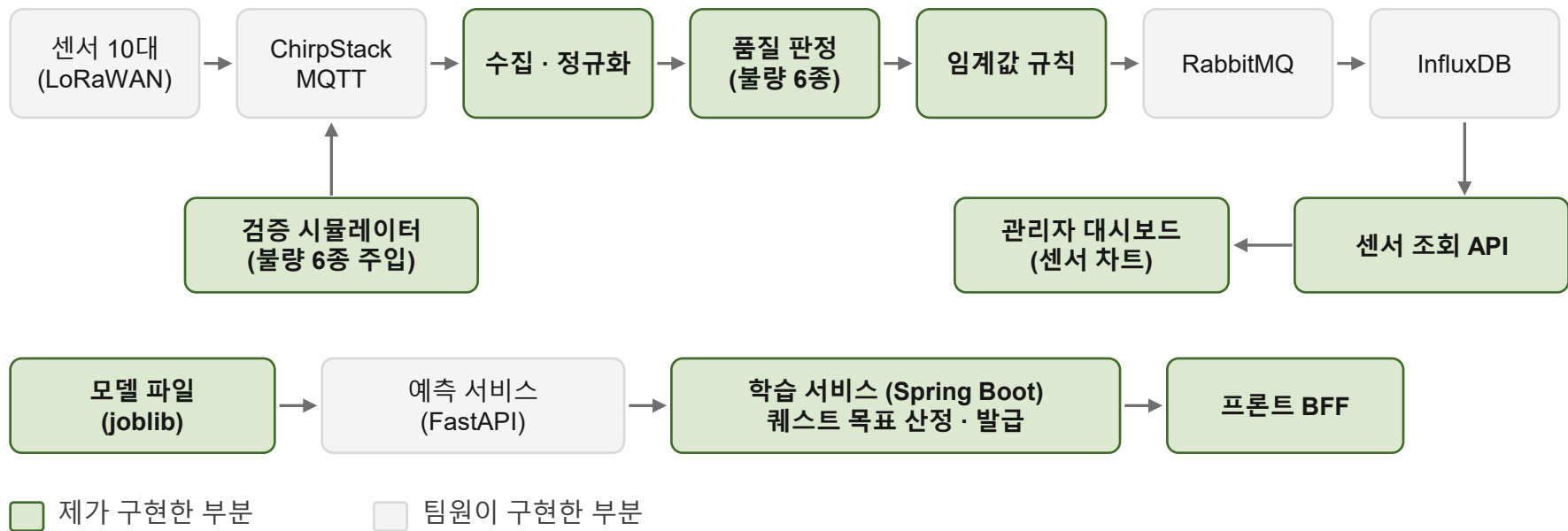
② 요구사항

- 기능:** 센서 데이터 수집 · 불량 6종 판정 · 임계값 알림, 관리자 센서 대시보드, 예측 기반 퀘스트 발급, AI 학습 코칭
- 비기능:** 메시지 유실 0건, 강제 종료 뒤 복구, 예측 서비스가 멈춰도 퀘스트 발급

④ 기술 · 검증 기준

- 사용 기술:** Java 21 · Spring Boot · MQTT(Paho v5) · RabbitMQ · InfluxDB · Caffeine · Spring AI · FastAPI(예측 서비스) · JUnit 5 · Testcontainers · Docker
- 검증 기준:** 보낸 메시지 수와 받은 메시지 수가 같을 것, 주입한 불량 수와 탐지한 수가 같을 것, 발행부터 조회까지의 지연(p99), 강제 종료 뒤 복구 여부

③ 설계



Project

IoT 센서 데이터 파이프라인 구현 및 검증

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

- 1

수집 · 정규화 노드

ChirpStack이 보낸 메시지에서 센서 · 공간 · 측정값을 꺼내 공통 형식으로 바꿨습니다. 측정 시각은 게이트웨이 시계가 아닌 서버 수신 시각(nsTime) 중 가장 이른 값을 썼습니다.
- 2

품질 판정 · 임계값 규칙 노드

범위 밖 값(이상치), 같은 프레임 재수신(중복), 60초 넘는 도착(지연), 30분간 같은 값(무변동), 프레임 번호의 빈틈(결측), 측정 주기 3배의 침묵(끊김)을 가려 서로 다른 출력 포트로 보냈습니다. 이어서 관리자가 정한 임계값을 넘는 값은 룰 적중 이벤트로 따로 발행했습니다.
- 3

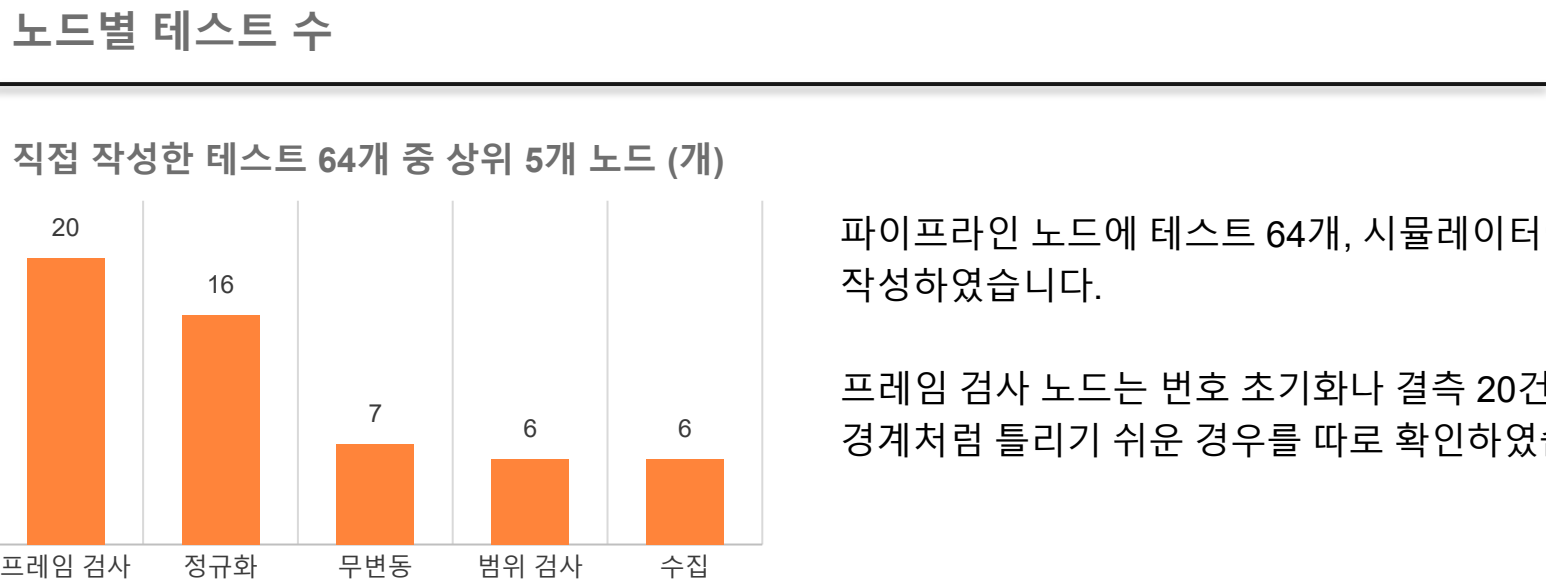
검증용 시뮬레이터

실제 센서는 불량이 언제 올지 알 수 없어, 6가지 불량을 정해진 수만큼 섞어 보내는 시뮬레이터를 따로 만들었습니다. 보낸 내역은 장부에 남겨 탐지 결과와 비교했습니다.
- 4

부하 · 지연 · 복구 시험

센서 50대가 1초마다 보내는 부하, 발행부터 InfluxDB 조회까지의 지연, 서비스를 강제로 종료한 뒤 다시 띄우는 상황을 시험했습니다.

검증 항목	결과
부하 (50대 × 1초)	보낸 32,855건 = 받은 32,855건
불량 6종 주입	주입한 수 = 탐지한 수
지연 (발행 → 조회)	p50 0.081초 · p99 0.124초
실제 운영 5시간 24분	8,963건 · 시각 오차 1.2초 이내



- 검증 결론
- 부하 · 불량 주입 시험에서 메시지를 한 건도 잃지 않고, 주입한 불량 6종을 모두 찾아내는 것을 확인하였습니다.
 - 실제 운영 데이터 8,963건에서 측정 시각 오차가 1.2초 이내로 유지되었습니다.

Project

예측 퀘스트 연동 및 AI 코칭 도구 구현

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

- 1

예측 기반 퀘스트 목표 (Spring Boot ↔ FastAPI)

FastAPI 예측 서비스가 준 내일 공부시간을 학습 서비스(Spring Boot)에서 3시간 30분 ~ 11시간 30분 사이로 맞춰 오늘의 퀘스트 목표로 정하고 발급하도록 구현하였습니다.
- 2

멈추지 않는 발급 (3단 폴백)

예측 서비스 호출이 실패하면 최근 7 등원일 평균으로, 그것도 없으면 기본값 3시간 30분으로 내려가 퀘스트가 항상 발급되도록 하였습니다. 공부 기록이 없는 사용자는 예측 서비스를 부르지 않습니다.
- 3

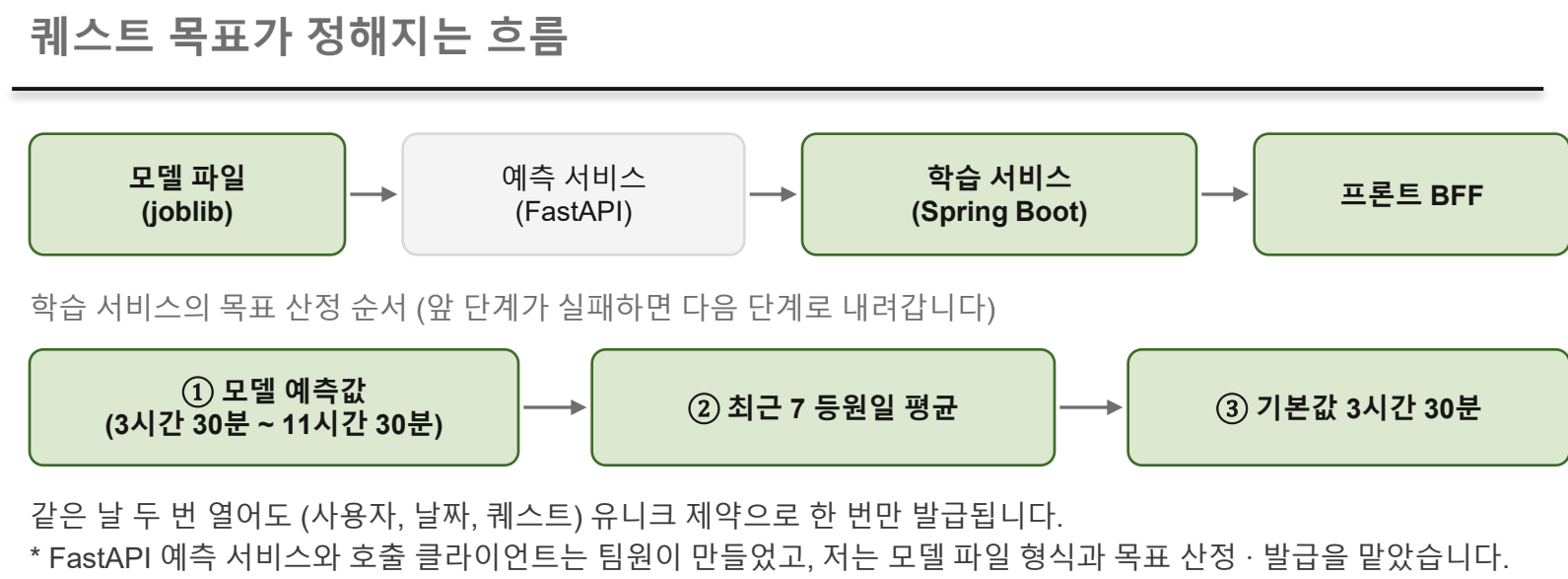
AI 학습 코칭 도구 (Spring AI)

학습 패턴 · 상위권 비교 · 주간 리포트 · 공간 환경 4개 도구를 Tool Calling으로 만들고, Gemini 키 여러 개를 돌려 쓰는 RoundRobinChatModel로 한도 초과(429) 시 다음 키로 넘겼습니다.
- 4

센서 대시보드 API (InfluxDB)

관리자가 공간별 센서 값을 일 · 주 · 월 단위로 볼 수 있도록 InfluxDB 시계열 조회 API와 프론트 BFF, 관리자 대시보드의 센서 차트를 만들었습니다.

통합 테스트로 확인한 경우	결과
예측 서비스 정상	예측값으로 발급 (상 · 하한 적용)
예측 서비스 실패	최근 7 등원일 평균으로 발급
공부 기록 없음	기본값 3시간 30분으로 발급
같은 날 다시 열기	다시 발급하지 않음



- 구현 결론
- 예측 서비스가 멈추거나 값을 주지 않아도 퀘스트 발급은 멈추지 않습니다.

· RoundRobinChatModel은 키 순환 · 429 전환 · 전체 쿼다운 등 10가지 경우를 단위 테스트로 확인하였습니다.

Troubleshooting

1 개발 배경

2 요구사항

3 설계

4 기술 · 검증 기준

5 구현

6 검증

7 문제 해결

8 성찰

CASE 01

PROBLEM

운영 데이터를 4시간 관측하니, 센서 9대에서 측정 시각이 수신 시각보다 정확히 242초 앞선 메시지가 섞여 들어왔습니다.

CAUSE

메시지가 지나온 경로를 하나씩 추적해, 중간 단계(Node-RED)가 틀어진 게이트웨이 한 대의 시계를 측정 시각으로 쓰는 것을 찾았습니다.

SOLUTION

242초를 빼서 맞추면 실제 지연까지 가려지므로, 중간 단계를 없애고 서버 수신 시각을 썼습니다. 이후 8,963건에서 재발하지 않았습니다.

CASE 02

PROBLEM

서비스를 강제 종료한 뒤 다시 띄우자 결측 신고가 10분 동안 반복되고, 정상 데이터가 저장소로 한 건도 넘어가지 않았습니다.

CAUSE

빠진 프레임이 많을 때 요약 신고 후 바로 return하는 분기가, 마지막 프레임 번호 갱신과 다음 노드 전달을 건너뛰고 있었습니다.

SOLUTION

return을 없애고 두 경로 모두 프레임 번호를 갱신하도록 고친 뒤 회귀 테스트 4개를 추가하였습니다. 재시험에서 받은 데이터가 모두 저장되었습니다.

CASE 03

PROBLEM

같은 사용자가 퀘스트 화면을 동시에 열면 오늘의 퀘스트가 두 번 발급될 수 있다는 코드 리뷰를 받았습니다.

CAUSE

발급 여부를 조회한 뒤 저장하는 사이에 다른 요청이 끼어들 수 있었고, JPA saveAll은 제약 위반이 나면 트랜잭션을 되살릴 수 없었습니다.

SOLUTION

ON CONFLICT DO NOTHING으로 중복 발급을 예외 없이 걸러 내고, Testcontainers 통합 테스트로 한 행만 남는 것을 확인하였습니다.

Retrospect

잘한 것

- 기다려서는 확인할 수 없는 불량 데이터를 시뮬레이터로 재현해, 판정 로직을 수치로 검증하였습니다.
- 예측 서비스가 멈춰도 퀘스트가 발급되도록 3단 폴백을 두었습니다.

아쉬운 것

- 마지막 프레임 번호를 메모리에만 두어, 재시작 직후 빠진 데이터 5,383건 중 3,232건을 결측으로 신고하지 못했습니다.
- 품질 판정 기준(지연 60초 · 무변동 30분)을 코드에 고정해, 운영 중에 바꾸려면 다시 배포해야 했습니다.

다시 한다면

- 프레임 번호를 외부 저장소에 두어, 재시작 뒤에도 이어서 결측을 판정하겠습니다.
- 퀘스트가 어떤 경로(모델 · 규칙 · 기본값)로 발급됐는지 비율을 집계해, 예측 서비스의 상태를 지켜보겠습니다.

함께 일한 방식

- 매일 아침 데일리 스크럼을 통해 서로의 진행 상황을 공유하는 시간을 가졌습니다.
- 매주 금요일 KPT 회고 시간을 통해 유지해야 할 것과 문제점, 개선해야 할 점을 논의하고 다음 주에 적용하였습니다.
- 모든 코드는 PR 리뷰를 거쳐 병합하였고, 리뷰에서 받은 의견은 코드와 테스트에 반영하였습니다.
- 각 파트별로 요구사항이나 결정사항을 문서화하여 Notion과 GitHub에 기록하고 공유할 수 있도록 하였습니다.

Project 02

Spring AI Library

자연어 도서 검색 · 추천 서비스

2026.06 ~ 2026.07 · 4인 (성능 · 데이터 파트)

GitHub

<https://github.com/AIoT-3/Spring-Ai-Library-Team6>

맡은 역할

리뷰 통계 · AI 요약

이벤트 · RabbitMQ 큐 · 누적 요약

리뷰의 추천 반영

IN 쿼리 한 번 · 프롬프트 규칙

시맨틱 캐싱

pgvector 코사인 유사도 · TTL · 워업

Project

자연어 도서 검색 · 추천 서비스 (Spring AI Library)

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

① 개발 배경

- 키워드가 정확히 맞아야 찾을 수 있는 기존 도서 검색과 달리, 자연어로 물어도 의미가 비슷한 책을 찾아 추천하는 서비스를 만들었습니다.
- 저는 성능 · 데이터 파트를 맡아, 독자 리뷰를 추천 근거로 쓰고 비슷한 질문에는 LLM을 다시 부르지 않도록 하는 일을 맡았습니다.

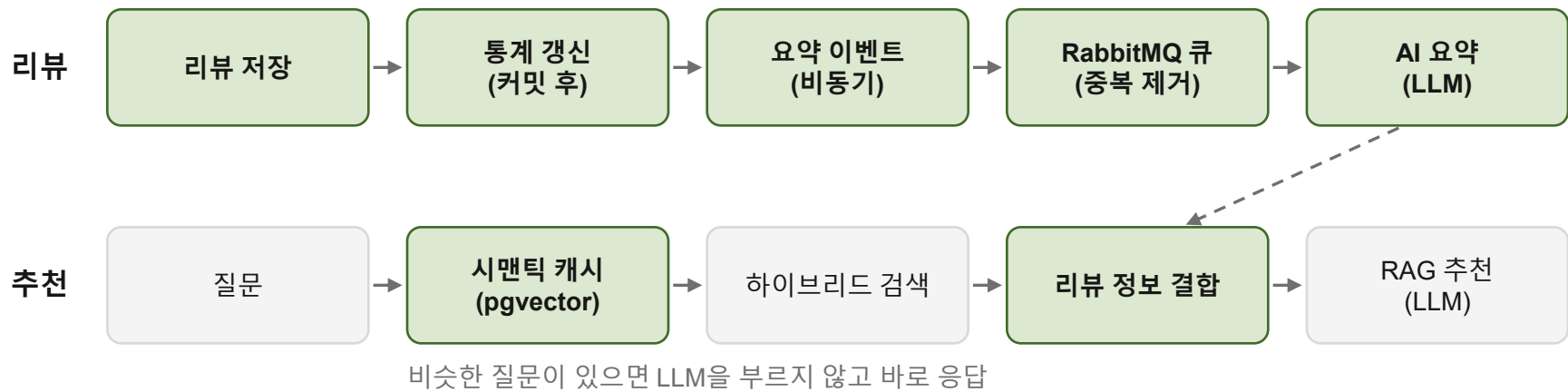
② 요구사항

- 기능:** 리뷰 저장 시 평점 통계 갱신, 리뷰 AI 요약, 추천에 평점 · 리뷰 요약 반영, 비슷한 질문의 답 재사용
- 비기능:** LLM 요약이 리뷰 저장 응답을 늦추지 않을 것, 같은 책의 요약이 중복 생성되지 않을 것

④ 기술 · 검증 기준

- 사용 기술:** Java 21 · Spring Boot · Spring AI · Spring Data JPA · PostgreSQL · pgvector · RabbitMQ · Ollama(qwen3.5:9b · bge-m3)
- 검증 기준:** 리뷰 저장부터 요약까지 순서대로 실행될 것, 같은 책의 요약이 한 번만 만들어질 것, 캐시 적중 시 LLM을 부르지 않을 것

③ 설계



■ 제가 구현한 부분 ■ 팀원이 구현한 부분

Project

구현 과정 및 검증

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

- 1

커밋 뒤에 실행되는 통계 갱신

리뷰 저장 트랜잭션이 커밋된 뒤에만 통계를 갱신하도록 AFTER_COMMIT 이벤트 리스너를 두고, 갱신은 새 트랜잭션(REQUIRES_NEW)에서 저장하였습니다.
- 2

큐로 분리한 AI 요약

요약은 LLM 호출이라 오래 걸려 RabbitMQ 큐에 작업만 넣고, 리스너가 3~5개 스레드로 따로 처리하게 하였습니다. 리뷰 저장 응답은 요약을 기다리지 않습니다.
- 3

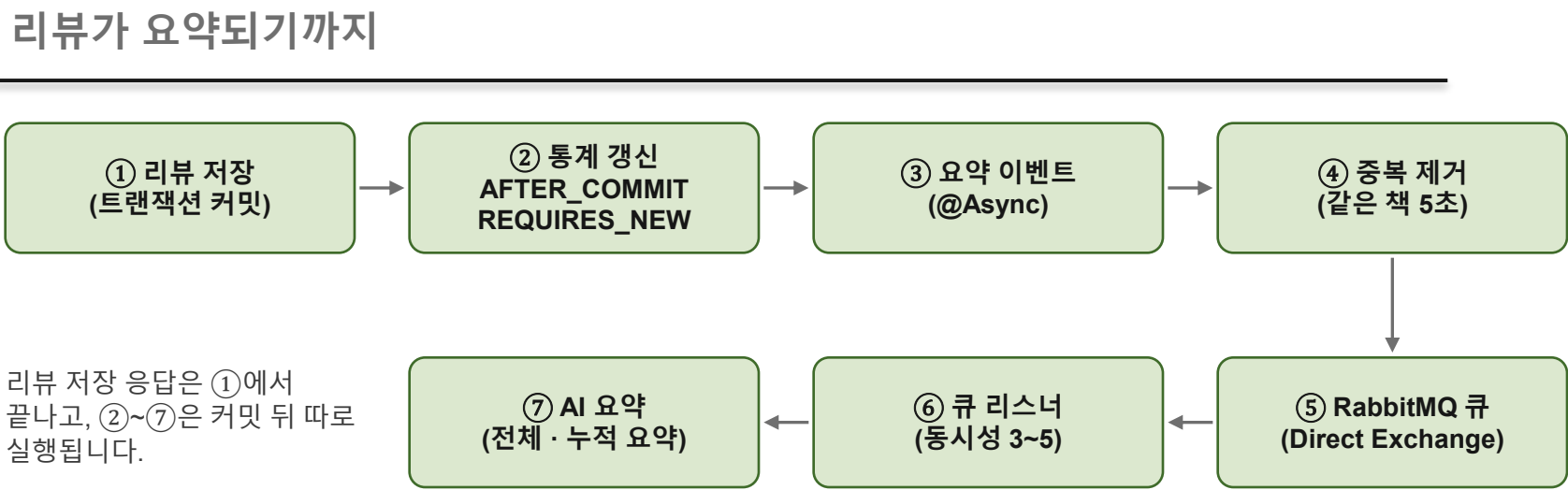
리뷰 정보의 추천 반영

후보 도서의 리뷰 요약에 IN 쿼리 한 번으로 모아 N+1 조회를 막고, 평점 4.0 이상 · 리뷰 20개 이상인 도서를 우선하는 규칙을 프롬프트에 넣었습니다.
- 4

시맨틱 캐싱

질문을 임베딩(bge-m3, 1024차원)해 pgvector에서 코사인 유사도 0.95 이상인 이전 질문을 찾으면, LLM을 부르지 않고 저장해 둔 답을 돌려줍니다.

장치	설계 값
큐 중복 제거	같은 책 요청은 5초 안에 1건
요약 실행 조건	리뷰 5개 이상 · 변경 있음 · 생성 중 아님
전체 재요약 기준	새 리뷰가 30% 이상일 때
캐시 유사도 · 유지	0.95 이상 · 30분 (매시 정리)



- ### 검증 결론
- 리뷰 10개를 넣는 시드 테스트로 통계 갱신 → 큐 적재 → 요약이 순서대로 실행되는 것을 로그로 확인하였습니다.
 - 앱을 시작할 때 인기 질문 8개를 미리 캐시에 넣어, 첫 질문부터 캐시를 쓸 수 있게 하였습니다.

Troubleshooting

1 개발 배경

2 요구사항

3 설계

4 기술 · 검증 기준

5 구현

6 검증

7 문제 해결

8 성찰

CASE 01

PROBLEM

리뷰가 커밋된 뒤 통계를 갱신하도록 AFTER_COMMIT 리스너를 두었는데, 리스너에서 저장한 통계가 DB에 반영되지 않았습니다.

CAUSE

AFTER_COMMIT 시점에는 원래 트랜잭션이 이미 커밋을 마쳐, 기본 전파(REQUIRED)로 참여한 쓰기는 다시 커밋되지 않았습니다.

SOLUTION

리스너에 REQUIRES_NEW를 적용해 통계 갱신을 새 트랜잭션에서 커밋하고, 커밋이 끝난 뒤에 요약 이벤트를 발행하도록 순서를 정하였습니다.

CASE 02

PROBLEM

리뷰가 짧은 시간에 몰리면 같은 책의 요약 작업이 여러 번 큐에 들어가, 같은 요약이 여러 번 만들어졌습니다.

CAUSE

리뷰마다 요약 이벤트가 발행되고, 큐 리스너가 3~5개 스레드로 같은 책의 작업을 동시에 처리했기 때문입니다.

SOLUTION

5초 안의 같은 책 요청은 한 건으로 합치고, 실행 직전 '생성 중' 표시를 먼저 저장하였으며, @Version 낙관적 락으로 동시 수정을 막았습니다.

CASE 03

PROBLEM

리뷰가 추가될 때마다 모든 리뷰를 다시 요약하면, 리뷰가 많은 책일수록 LLM 호출 비용이 계속 커졌습니다.

CAUSE

요약이 매번 리뷰 전체를 처음부터 다시 읽는 구조였기 때문입니다.

SOLUTION

직전 요약 이후의 새 리뷰만 요약해 기존 요약과 합치고, 새 리뷰가 30% 이상일 때만 10개씩 나눠 전체를 다시 요약하도록 하였습니다.

Retrospect

잘한 것

- 오래 걸리는 LLM 요약물 큐로 분리해, 리뷰 저장 응답이 요약물 기다리지 않게 하였습니다.
- 같은 책의 요약이 중복되지 않도록 큐 단계와 DB 단계에서 함께 막았습니다.

아쉬운 것

- 캐시 유사도 0.95와 유지 시간 30분을 실제 측정 없이 정하였습니다.
- 요약에 실패한 작업을 다시 시도하는 장치 없이 로그만 남겼고, 요약 품질도 수치로 평가하지 못하였습니다.

다시 한다면

- 캐시 적중률과 응답 시간을 측정해 유사도 기준과 유지 시간을 정하겠습니다.
- 실패한 요약 작업은 재시도 큐(DLQ)로 보내 다시 처리하겠습니다.

함께 일한 방식

- 검색 엔진 · RAG · 성능 데이터 · 인터페이스 네 도메인으로 나뉘, 도메인마다 패키지를 맡아 개발하였습니다.
- 제가 만든 시맨틱 캐시에 팀원이 개인화 요청은 캐시를 거치지 않는 조건을 더해, 사용자마다 다른 결과가 섞이지 않게 하였습니다.
- 제 파트의 설계와 결과는 개인 보고서로 정리해 저장소에 올려 팀과 공유하였습니다.

Project 03

Mini Dooray

프로젝트 관리 협업 툴 · 4개 서비스 MSA

2026.05.15 ~ 2026.05.22 (8일) · 4인

GitHub

<https://github.com/AIoT-3/minidooray-team3-account-api>

맡은 역할

회원 API 설계 · 구현

가입 · 조회 · 수정 · 탈퇴 · 휴면 전환

예외 처리

도메인 예외 5종 → 404 · 409 · 401

테스트 · 품질

테스트 40개 · 커버리지 100% · SonarQube

Project

프로젝트 관리 협업 툴 (Mini Dooray)

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

① 개발 배경

- NHN Dooray!를 본떠, 하나의 앱에 모든 기능을 넣지 않고 Front · Gateway · Account API · Task API 4개 서비스로 나눈 협업 툴을 만들었습니다.
- 저는 회원 정보를 관리하는 Account API를 맡았습니다.

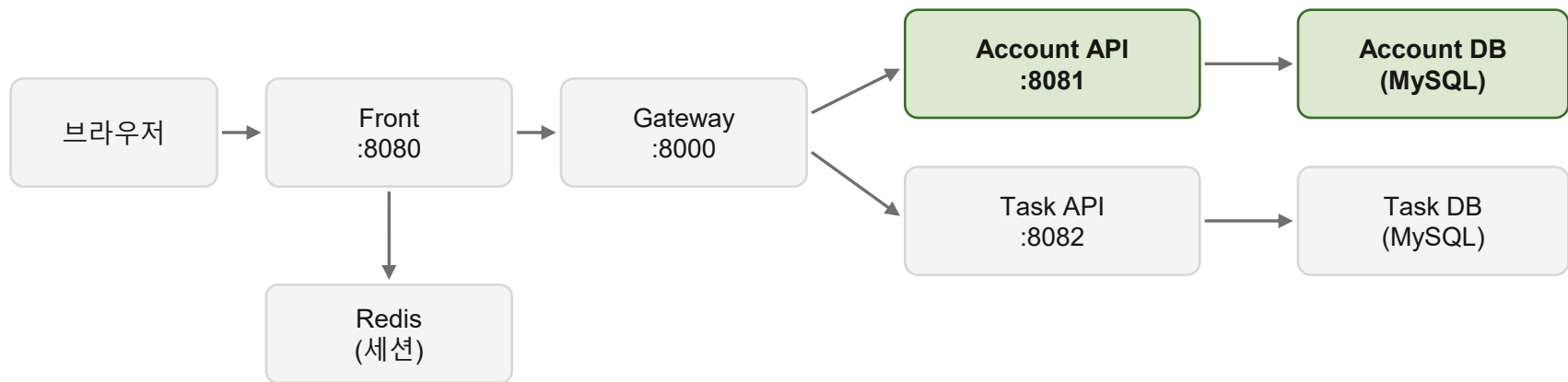
② 요구사항

- 기능:** 회원가입 · 회원 조회 · 이름 검색 · 정보 수정 · 탈퇴 · 휴면 전환
- 비기능:** 오류마다 알맞은 상태 코드와 같은 형식의 오류 응답, 계층별 테스트와 커버리지 측정

④ 기술 · 검증 기준

- 사용 기술:** Java 21 · Spring Boot · Spring Data JPA · MySQL · H2 · JUnit 5 · Mockito · JaCoCo · SonarQube
- 검증 기준:** Repository · Service · Controller 계층별 테스트, JaCoCo 명령어 · 라인 · 분기 커버리지

③ 설계



서비스마다 DB를 나누고, 서비스 사이는 FK 없이 account_id 값으로만 연결하였습니다. (팀 공통 설계)

■ 제가 구현한 부분 ■ 팀원이 구현한 부분

Project

구현 과정 및 검증

- 1 개발 배경
- 2 요구사항
- 3 설계
- 4 기술 · 검증 기준
- 5 구현
- 6 검증
- 7 문제 해결
- 8 성찰

- 1

API 명세 · ERD 설계

회원 엔드포인트의 URL과 요청 · 응답 형식을 먼저 정해 팀과 합의하고, 회원 테이블의 ERD를 설계하였습니다.
- 2

계층별 구현

Controller · Service · Repository로 나눠 가입 · 조회 · 수정 · 탈퇴 · 휴면 전환을 구현하였습니다. 탈퇴와 휴면은 행을 지우지 않고 회원 상태만 바꾸도록 하였습니다.
- 3

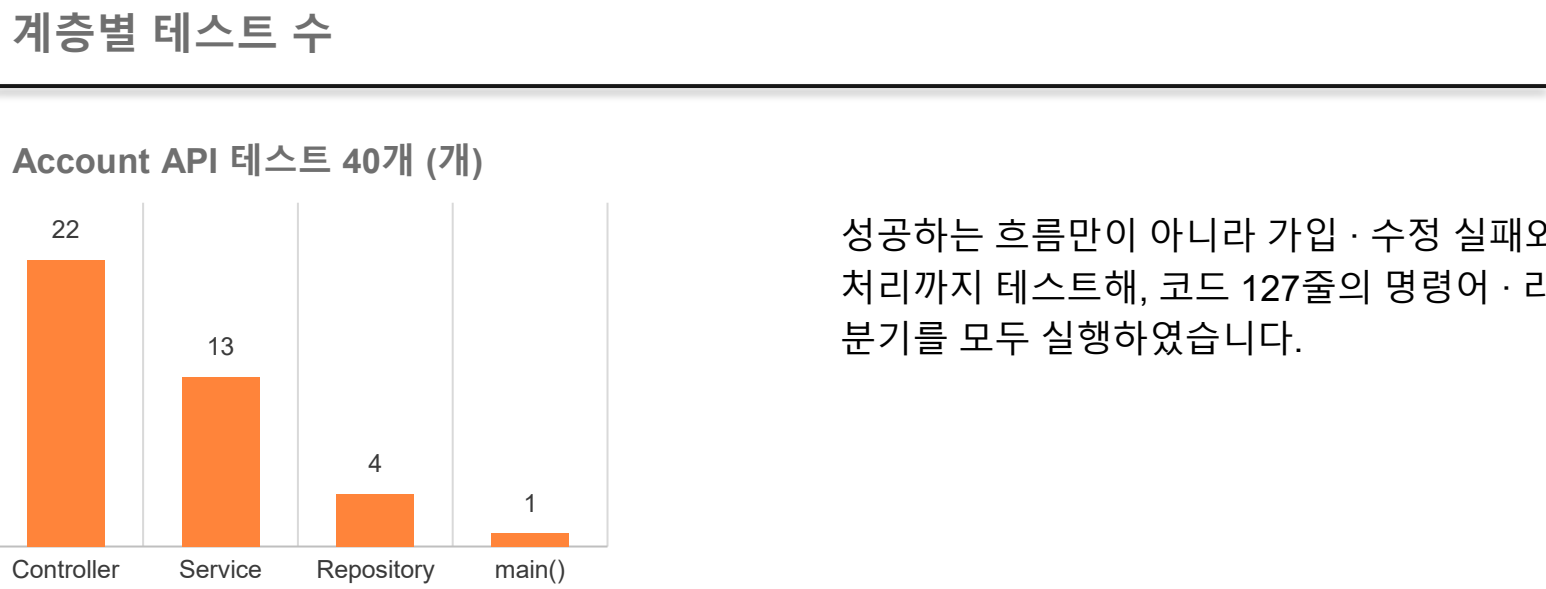
예외 처리

원인별 도메인 예외 5종을 만들고 @RestControllerAdvice에서 404 · 409 · 401로 나눠, 같은 형식의 오류 응답으로 내려 주었습니다.
- 4

계층별 테스트 · 품질 측정

Repository · Service · Controller 테스트 40개를 작성하고, JaCoCo로 커버리지를, SonarQube로 코드 품질을 측정하였습니다.

커버리지 (JaCoCo)	결과
명령어	556 / 556 (100%)
라인	127 / 127 (100%)
분기	16 / 16 (100%)
테스트 실행	43건 실행 · 실패 0건



- 검증 결론
- 실패하는 경우와 애플리케이션 시작 코드까지 테스트해 커버리지 100%를 달성하였습니다.
 - 없는 회원은 404, 중복은 409, 비밀번호 불일치는 401로 응답해 호출하는 쪽에서 원인을 구분할 수 있게 하였습니다.

Troubleshooting

1 개발 배경

2 요구사항

3 설계

4 기술 · 검증 기준

5 구현

6 검증

7 문제 해결

8 성찰

CASE 01

PROBLEM

비밀번호가 틀려도 '없는 회원'과 같은 404가 응답되어, 호출하는 쪽에서 오류 원인을 구분할 수 없었습니다.

CAUSE

서비스가 IllegalArgumentException 같은 일반 예외를 던져, 핸들러는 예외 종류마다 상태 코드를 하나씩만 정할 수 있었습니다.

SOLUTION

원인별 도메인 예외 5종을 만들고 @RestControllerAdvice에서 404 · 409 · 401로 나눠, 같은 형식의 오류 응답으로 내려 주었습니다.

CASE 02

PROBLEM

JaCoCo로 측정해 보니 main() 메서드와 예외 처리, 실패 분기가 테스트되지 않은 채 남아 있었습니다.

CAUSE

가입 · 조회처럼 성공하는 흐름 위주로 테스트를 작성해, 실패하는 경우와 애플리케이션 시작 코드가 빠져 있었습니다.

SOLUTION

가입 · 수정 실패와 예외 처리, 빈 값 입력 테스트를 추가하고 main()은 MockedStatic으로 확인해 명령어 · 라인 · 분기 커버리지 100%를 달성하였습니다.

CASE 03

PROBLEM

SonarQube로 분석하니 같은 오류 메시지 문자열이 메서드마다 반복되고, 테스트의 기대값과 실제값 순서가 뒤바뀐 코드가 지적되었습니다.

CAUSE

메서드를 구현할 때마다 같은 코드를 복사해 쓰고, assertEquals의 인자 순서를 신경 쓰지 않았기 때문입니다.

SOLUTION

메시지를 상수로 모으고 결과를 바로 반환하도록 정리했으며, 테스트는 기대값을 앞에 두도록 고쳤습니다. 메시지는 이후 도메인 예외로 옮겼습니다.

Retrospect

잘한 것

- API 명세를 먼저 정해, 네 서비스가 서로를 기다리지 않고 동시에 개발할 수 있었습니다.
- 실패하는 경우까지 테스트해 커버리지 100%를 달성하였습니다.

아쉬운 것

- 커버리지 100%는 코드 127줄 규모에서 나온 값이라, 기능이 늘어나면 유지하기 어렵습니다.
- 회원 목록 조회에 페이징과 검색 조건이 부족하였습니다.

다시 한다면

- 커버리지 숫자보다 실패 시나리오를 먼저 정하고 테스트를 설계하겠습니다.
- 회원 목록 조회에 페이징을 적용하겠습니다.

함께 일한 방식

- ERD와 REST API 명세는 팀 전원이 함께 설계하고, 저장소를 서비스 단위로 나눠 역할을 분담하였습니다.
- PR 리뷰로 코드 품질을 함께 관리하고, 리뷰에서 나온 경로 · 설정 오류는 통합 단계에서 고쳤습니다.

범죄 심각도 지수 (KCSI)

2024.11 ~ 2025.05 · 6인 · KCI 논문 공동저자

① 프로젝트 소개

살인과 절도를 같은 1건으로 세는 기존 통계의 한계를 보완하려고, 범죄 유형별 형량을 가중치로 쓴 한국형 범죄 심각도 지수를 설계하고 사회·환경 요인으로 얼마나 설명되는지 검증하였습니다.

② 사용 기술

R · SPSS · 다중회귀 · Ridge · Lasso · ElasticNet

③ 맡은 역할

- 6개국 범죄 심각도 지수의 가중치 근거를 비교하였습니다.
- KOSIS 데이터를 수집하고 대법원에 형량 자료를 요청해 가중치를 만들었습니다.
- 상관분석부터 규제회귀까지 분석을 진행하였습니다.

개인 맞춤형 퀘스트 (공부시간 예측)

2026.07 ~ 09 · 8인 (AI 파트 2인)

① 프로젝트 소개

어제까지의 기록으로 내일 공부시간을 예측하는 회귀 모델을 만들고 FastAPI 예측 서비스로 서빙해, 예측한 공부시간을 개인 맞춤 퀘스트로 주는 기능입니다.

② 사용 기술

Python · Ridge · Random Forest · XGBoost · LightGBM · FastAPI

③ 맡은 역할

- 가상 데이터 60명 × 300일을 설계하고 무결성 검사 23개로 검증하였습니다.
- 피처 32개를 만들고 모델 4종을 같은 시간 분할로 비교하였습니다.
- LightGBM으로 기존 규칙보다 오차(MAE)를 82.5분 줄였습니다.

사계절 붕괴와 에너지 수요

2025.05 ~ 06 · 3인 · 경진대회 출전

① 프로젝트 소개

봄·가을이 짧아지고 여름·겨울이 길어지는 변화가 서울과 부산의 전력 수요에 미친 영향을 검정하고 회귀로 설명한 분석입니다.

② 사용 기술

R · SPSS · t-검정 · Kruskal-Wallis · 다중회귀

③ 맡은 역할

- 기상청 계절 개시일 기준으로 계절·폭염·한파 일수를 계산해 변수를 만들었습니다.
- 정규성이 깨지자 비모수 검정으로 바꿔 계절 간 차이를 확인하였습니다.
- 계절 더미 모형으로 서울 test Adj-R²를 0.385에서 0.492로 높였습니다.

감사합니다.

권세윤

010-3616-4913

tpdbs3616@naver.com

<https://github.com/sy-103>